

AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 9.1: The Agent Opportunity — From Chatbox to Agent Swarms

Zhigang Feng

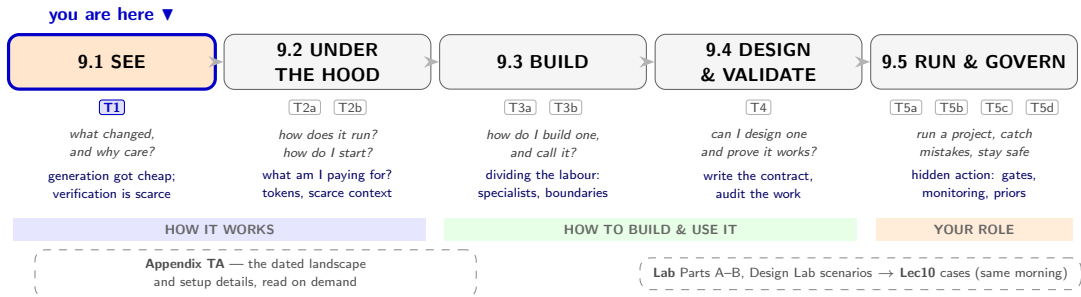
2026-09-12 15:58:56

Six Minutes, One Figure

Placeholder — written in Phase B. The specification is in the source comment above this frame.

Lecture 9 in One Map

One lecture, five parts, eleven decks, three questions: how agentic AI works, how to use it well, and what your role is.



Two research threads run through all five parts: **HA-Ramsey** (a published research program, full case in Lec10 T8) and the **ES Fellows dataset** (a project you could build this week, full case in Lec10 T4). Here we study the workflows; the cases themselves come later this morning.

The Agent Opportunity

What changed, and why an economist should care

Why Economists Should Care: Execution Is the Bottleneck

The bottleneck in applied research is often not ideas. It is execution.

- Turning a good question into clean code, diagnostics, tables, and figures is slow
- Many research tasks are repetitive: reshape data, rerun specifications, trace sources, refactor scripts, update plots
- Agentic tools compress the time from *idea* to *first defensible result*
- That raises the return to the skills economists uniquely provide: question design, institutional knowledge, validation, and judgment

The promise is not “AI does research for you.” It is faster implementation and iteration around a question you still define and defend. The rest of the lecture supplies what that takes: how an agent runs (9.2), how to build and call one (9.3), how to design and validate one (9.4), and how to run, check, and govern a whole project (9.5).

The Ladder: From a Chat Window to a Verified Swarm

Placeholder — written in Phase B. The specification is in the source comment above this frame.

Delegating to an Agent Is a Principal–Agent Problem

Placeholder — written in Phase B. The specification is in the source comment above this frame.

Placeholder — written in Phase B. The specification is in the source comment above this frame.

Story 1 — HA-Ramsey: Weeks, Not Years

Proper use of an agentic workflow turned one published solver into a new research program.

Seed: a working, validated representative-agent (RA) “deep Ramsey” neural solver — the complete code behind Feng–Han–Zhu (2026).

Target: the heterogeneous-agent (HA) Ramsey model of Chien–Feng–Wen (2027) — the entire paper: idiosyncratic risk, borrowing constraints, a 4–5D state.

Outcome: a full HA research codebase *plus* three methodological discoveries that surfaced from the AI iteration itself — α -shapes for non-convex feasible sets, the admissible set as a viability kernel, and a Lyapunov-type penalty for saddle-path selection.

- The AI supplied **throughput** — LaTeX drafting, PyTorch translation, diagnostics, 32+ sweeps; “time from seed to working solver: **weeks, not years**; the binding constraint was economist judgment at the milestones, not implementation labor”

Why show this first: this is what “using it properly” buys. Not autocomplete — research.

Full case with the economics and the figures: Lec10 T8, later this morning. Source: HA-Ramsey case-study package.

Story 2 — 882 Fellows: A Dataset You Could Build This Week

The second thread is deliberately ordinary: build a credible dataset about the economics profession from the public web.

- **Task:** career metadata for all 882 current Econometric Society Fellows — election year, PhD year, research fields, career stage
- **Seed:** the official Society roster, split into 18 batches of 50; every filled field must carry a source URL from an official or institutional page
- **What makes it hard:** roster control, source discipline, state across sessions, auditable provenance — *bookkeeping, not intelligence*
- **What happened:** the agentic workflow scaled the search massively; the honest headline was about what still could *not* be measured (T5c tells that part)

Why two stories: HA-Ramsey shows the ceiling of the method. The fellows dataset is the floor you can stand on immediately — in Exercise B, you build one batch of 50 yourself.

Full case with the coverage results: Lec10 T4, later this morning.

Chat, Copilot, Agent

Where the loop runs, and what it can touch

What Modern Web Chat Already Does Well

As of mid-2026, a strong web-chat product is already useful for economists.

- Brainstorming model setups, empirical strategies, review rubrics, and robustness checks
- Summarizing papers, referee reports, and lecture notes
- Inspecting uploaded files and explaining code snippets
- Sometimes browsing the web or running limited sandboxed tools
- Helping you draft a specification before you open your editor

So the weak baseline is not “a browser cannot do anything.” The weak baseline is a *single chat session without a durable harness*.

What Breaks Without a Harness: The Fellows Test

Try the fellows task in a plain chat window. It is a job of many sittings — and four things break within the first hour.

1. **Durable context is weak.** The task spans days of separate sessions. The coding rules you settle today — which sources count, how to record a missing PhD year — are not reliably remembered by tomorrow's fresh chat
2. **Reusable workflow is weak.** Your checklist lives in prompts you retype (slightly differently) each time, not in a file the tool reads automatically
3. **Controlled execution is weak.** The chat cannot open the roster file, write the CSV, or rerun a check — *you* become the copy-paste machine between the model and your folder
4. **Audit trail is weak.** Six weeks later: which source URL backed the PhD year of fellow #312? A scrolled-away transcript cannot answer field-level questions

That is the real contrast. Agents are not magic because they are “smarter.” They are useful because they operate inside a structured project with files, tools, and repeatable workflows.

Chat vs. Agent: Same Brain, Different Arms

One key technical change made agents possible: modern LLMs (post-2023) support structured tool calls.

Instead of writing “you should run `python solve.py`,” the model emits a machine-readable request — something like `{"tool": "Bash", "command": "python solve.py"}` — and a **harness**, software installed on your machine, executes it and returns the real output. (T2a opens this loop in machine language.)

	claude.ai (browser)	Claude Code (terminal)
LLM model	the same frontier model	
Harness	none (sandboxed)	local CLI on your machine
Can run Python?	no	yes— <i>your</i> Python, <i>your</i> machine
Sees real errors?	no	yes—loops back automatically

Same brain, different arms. The terminal agent is not a magically smarter model. The harness is what flips the switch from text-only output to text + actions inside a project.

The Spectrum, and Our Featured Tool

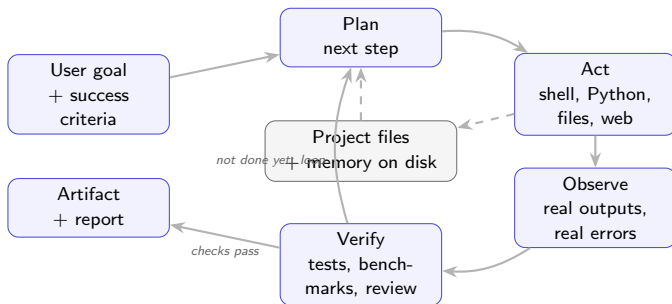
	Web chat	IDE assistant	Terminal agent
Best at	thinking, drafting	editing local code	end-to-end execution
Context	conversation + uploads	open files	full project tree
Typical role	architect's whiteboard	coding copilot	workflow operator
Economic example	design the specification	clean one regression file	run the full pipeline

Concrete example throughout this lecture: Claude Code, a terminal agent. **General lesson:** the same workflow logic appears in Codex CLI, Gemini CLI, Aider, and similar tools.

The only tool-comparison table in Lecture 9 — the workflow matters more than the tool. T5a returns to tool choice per pipeline step.

The Agentic Loop: Plan, Act, Observe, Verify, Repeat

An agentic workflow is not one long generation. It is a control loop wrapped around tools, files, and checks.



- **Orchestrator:** the loop deciding what to do next — plan, act, observe, verify
- **Memory:** saved project state on disk, not just chat history (dashed arrows: the loop reads it and writes it)
- **Verifier:** tests, benchmarks, or review checks deciding whether output survives

What Makes Something “Agentic”? Four Features

- **Tool use** — invokes pandas, curl, R, statsmodels, the shell, whatever the task requires
- **Autonomy** — plans a multi-step empirical workflow without re-prompting.
e.g., “download FRED → clean → regress → plot → save table” as one command
- **Iteration** — observes that a regression failed, identifies the bad column, fixes the formula, reruns
- **Persistence** — remembers your project conventions across sessions (variable names, preferred packages, validated benchmarks)

All four are cumulative. Tool use without autonomy = a glorified interactive console (a REPL — *read-eval-print loop*, like the Python `■>` prompt). Autonomy without iteration = a script that dies on the first error. You need all four for “agentic.”

The Landscape in Three Frames

Patterns, costs, and how long agents really work

Four Orchestration Patterns

Production agent systems are built from a small set of recurring control structures.

Pattern	Shape
Orchestrator–worker	a lead agent decomposes a task and dispatches workers; <i>dominant in production</i>
Planner–executor	one component plans, another executes step by step
Reflection	the agent critiques and revises its own output before returning
Handoff	control passes between specialized agents (e.g. triage → specialist)

The agent team you will build in Part 9.3 and run as an RA team in T5b is an orchestrator–worker system: the main agent orchestrates; code-reviewer, domain-reviewer, and verifier sub-agents are the workers. The landscape just adds vocabulary and tooling around the same loop.

The Multi-Agent Cost–Benefit Decision

More agents is not free. Treat it as marginal value vs. marginal cost.

- Anthropic reports its multi-agent research system beat a single-agent baseline by **+90.2%** on its internal research-eval. . .
- . . . at roughly **15× the token cost**
- So multi-agent pays off when the task is broad, parallelizable, and the answer's value is high relative to tokens
- It is wasteful for narrow, sequential, or cheap tasks

[solid, label: **Anthropic-internal-eval**] — a vendor benchmark on a self-defined research task, not an external suite. Read the +90.2% as directional, not portable.

Economist framing. You are choosing a point where marginal research value = marginal token (and latency, and debugging) cost. The Aiyagari sub-agent review in T5b is justified on *wall-clock*, not because “more agents are better.”

How Long Can an Agent Actually Work? Measure, Don't Trust Marketing

Vendors say “works for hours” or “runs for days.” METR gives a measured alternative.

- METR's *task-length-doubling* result: the task length an agent completes at **50% reliability** has doubled roughly every **7 months**, accelerating to **~4 months** in 2024–25. **[solid]**
- Time Horizon 1.1 (Jan 2026) pegs **Opus 4.5** $\approx$ **320 min** and **GPT-5** $\approx$ **214 min** (50%-reliability horizon).
- Use these measured minutes instead of vendor “multi-hour/multi-day autonomy.”

Metric fragility. The number is tied to a **50% reliability** threshold and a specific task suite; the horizon at 80% reliability is much shorter, and it shifts across suites. A measured-but-fragile number still beats a vendor claim.

[hype counterpoint] product pages advertising “multi-hour” or “multi-day” autonomy — no reliability threshold stated.

Research translation: the right question is not “can it run for hours?” but “at what reliability, on what task distribution, validated how?”

What This Means for Research

Four patterns, and what stays human

Four Research Patterns Across the Lec10 Cases

Pattern	Lec10 case(s)	Main risk if you use the wrong workflow
Reusable skill	Case 1: referee-style paper review	Generic comments, hallucinated quotes, missing quality gate
Empirical pipeline	Case 2: MEPS harmonization and figures	Wrong crosswalk, dropped survey weights, silent validation failure
Public-web dataset	Case 3: fellows metadata with field-level provenance	Weak sources, hidden missingness, no audit trail
Full-lifecycle collaborator	Cases 4–6: collaborator, 4-day sprint, HA-Ramsey	Polished output built on weak framing or claims discipline

Four patterns organize Lec10's six cases — and our two threads are instances: the fellows dataset (Case 3) and HA-Ramsey (Case 6, the full-lifecycle pattern at research scale).

Where RAG fits. Agentic RAG (Lec08 T5) differs from single-shot RAG: the agent decides *dynamically*, via the same tool-call mechanism, whether and when to retrieve. RAG is one tool among many that the agent can compose.

What Remains Human Across the Four Patterns

You still own

- the research question
- the success criteria
- the coding and source rules
- the validation target
- the final interpretation

The agent accelerates

- repetitive execution
- file handling and refactoring
- promptable decomposition
- reruns after fixes
- draft diagnostics and reports

Why this matters for economists: the gain is not that judgment disappears. The gain is that disciplined execution becomes cheaper.

Hold on to the left column. In 9.5 it becomes a job description: the researcher as project manager.

Division of Labor: You Provide, the Agent Provides

You provide	Agent provides
Question, identification logic, and source rules	Fast code generation and refactoring
Workflow choice and benchmark design	Boilerplate implementation and debugging
Knowledge of the data, model, and institutions	File edits, script writing, and test runs
Economic interpretation and seminar defense	Iteration on visible errors and outputs

Research version of the slogan: the agent can be a strong RA, but you still have to be the principal investigator.

Bridge: Open the Hood

Part 9.1 framed the terminal-agent workflow: chat alone is not enough; a harness wraps tools, files, and verifiers around the model so the loop can do real work.

- You have seen *what* agents do, *why* the harness exists, and what proper use produced in two real projects
- **Dated detail (Appendix TA):** the wider landscape — interoperability standards, frameworks, and product churn — read on demand
- **Next (9.2, T2a and T2b):** the question changes from “what is this?” to “*how does it actually work, and how do I start?*” — components, install, initialization, the CLI, and your first session

Arc: 9.1 See → 9.2 Under the Hood → 9.3 Build → 9.4 Design & Validate → 9.5 Run & Govern.